SmartKisan AI

Intelligent Crop Monitoring and Decision Support System

Engineering Project Documentation

Submitted by:

Krish - Full Stack Development

Krishna - IoT Engineering

Abhilasha - AI / Machine Learning

Project Duration:

3 Weeks

Date: October 26, 2023

Executive Summary

1.1 Project Overview

SmartKisan AI is an integrated agricultural technology solution designed to bridge the gap between traditional farming methods and modern digital precision agriculture. By leveraging the Internet of Things (IoT), Artificial Intelligence (AI), and web technologies, the system provides real-time monitoring of soil conditions and intelligent disease detection for crops. The platform aims to empower farmers with data-driven insights to optimize yield and reduce crop wastage.

1.2 Problem Statement

Agriculture faces significant challenges due to unpredictable weather patterns, soil degradation, and pest infestations. Traditional farming relies heavily on manual observation and historical knowledge, which often leads to:

- Delayed detection of crop diseases, resulting in significant yield loss.
- Inefficient use of water and fertilizers due to lack of real-time soil data.
- Inability to predict environmental impact on crop health accurately.

1.3 Project Objectives

The primary objectives of the SmartKisan AI project are:

1. **Real-time Monitoring:** To design an IoT-based system that continuously monitors critical soil parameters such as moisture, temperature, humidity, and nutrient levels (NPK).
2. **Intelligent Disease Detection:** To develop a Convolutional Neural Network (CNN) based AI model capable of analyzing crop leaf images to detect diseases with high accuracy.

3. **Decision Support System:** To create a user-friendly web interface that aggregates sensor data and AI predictions to provide actionable recommendations to farmers.

1.4 Expected Outcomes

Upon completion, the system is expected to deliver a functional prototype that allows farmers to remotely monitor their field conditions via a dashboard. The AI component will enable early diagnosis of plant diseases through simple image uploads, potentially reducing crop loss by providing timely treatment recommendations. The integrated system will serve as a foundational model for smart farming applications.

Team Contribution Distribution

The success of the SmartKisan AI project relied on the specialized skills of each team member. Tasks were distributed based on expertise in Full Stack Development, Embedded Systems (IoT), and Artificial Intelligence.

2.1 Contribution Summary

Team Member	Role	Key Responsibilities	Contribution
Krish	Full Stack Developer & System Integrator	Web architecture, UI/UX, API development, Cloud deployment	**40%**
Krishna	IoT Engineer	Sensor hardware setup, microcontroller programming, data telemetry	**35%**
Abhilasha	AI / ML Developer	Dataset curation, CNN model architecture, training, and evaluation	**25%**

2.2 Detailed Role Descriptions

Krish - Full Stack Development

Krish was responsible for the end-to-end development of the web application. This included designing the user interface to ensure accessibility for non-technical users (farmers), developing the backend architecture to handle data requests, and integrating the AI and IoT modules into a unified dashboard.

Krishna - IoT Engineering

Krishna focused on the hardware implementation. Responsibilities included selecting appropriate sensors for agricultural monitoring, wiring the microcontroller circuits, and writing the C++ firmware to read sensor data efficiently. He also handled the data transmission protocols to send local sensor data to the cloud database.

Abhilasha - AI / Machine Learning

Abhilasha led the intelligence module of the project. She gathered and preprocessed a large dataset of plant leaves, designed a custom Convolutional Neural Network (CNN), and trained the model to recognize various plant diseases. Her role ensured the system could provide accurate diagnostic capabilities.

Week 1 Progress: Planning & Initial Development

The first week focused on requirement analysis, system architecture design, and setting up the foundational components for each module.

3.1 Web Development (Krish)

Objective: Establish the frontend structure and user interface designs.

- Designed the **SmartKisan AI website layout** focusing on simplicity and readability.
- Created the initial **user interface (UI)** wireframes for the dashboard and landing page.
- Developed HTML5/CSS3 templates for the input forms where farmers will enter crop details.
- Implemented the **image upload functionality** required for the disease detection module.

Technologies: HTML5, CSS3, JavaScript, Figma (for design).

3.2 IoT Development (Krishna)

Objective: Hardware selection and sensor calibration.

- Researched and selected cost-effective sensors: **DHT11 (Temp/Humidity)** and **Capacitive Soil Moisture Sensor**.
- Configured the **ESP8266 NodeMCU** microcontroller as the main processing unit.
- Established circuit connections on the breadboard for initial testing.
- Performed baseline calibration of the soil moisture sensor using dry and wet soil samples.

Tools: Arduino IDE, ESP8266, DHT11 Sensor, Jumper Wires.

3.3 AI / ML Development (Abhilasha)

Objective: Data preparation and model architecture design.

- Collected the **PlantVillage dataset** containing thousands of healthy and diseased leaf images.
- Performed data cleaning to remove low-quality images.
- Applied **image preprocessing techniques**: resizing to 224x224 pixels, normalization (scaling pixel values to 0-1 range).
- Designed the initial architecture for the **Convolutional Neural Network (CNN)** using Keras Sequential API.

Tools: Python, TensorFlow, Keras, NumPy, Pandas.

Week 2 Progress: Core System Development

Week 2 was dedicated to the core logic implementation, data processing, and initial testing of individual modules.

4.1 Frontend & Backend Logic (Krish)

Activities:

- Developed the **Farmer Dashboard** to visualize data dynamically.
- Created RESTful API endpoints to receive data from the IoT devices.
- Designed the result page to display AI prediction confidence scores and treatment recommendations.
- Improved UI responsiveness to ensure the site works on mobile devices, which is critical for field usage.

4.2 IoT Programming & Data Processing (Krishna)

Activities:

- Wrote the embedded C++ code to read analog and digital signals from sensors.
- Implemented a **data acquisition loop** that reads sensors every 5 seconds.
- Added logic to average readings over 1 minute to reduce sensor noise.
- Formatted the output data into JSON strings for transmission.

Collected Data Metrics:

- Soil Moisture (%)
- Ambient Temperature (°C)
- Relative Humidity (%)

- Soil NPK values (Simulated for prototype)

4.3 Model Training & Evaluation (Abhilasha)

Activities:

- Implemented the CNN model:
  - 3 Convolutional layers with ReLU activation.
  - Max Pooling layers for dimensionality reduction.
  - Flatten and Dense layers for classification.
- Trained the model over **50 epochs** using a split of 80% training and 20% validation data.
- Monitored loss and accuracy curves to prevent overfitting.
- Achieved an initial validation accuracy of **88%**.

Week 3 Progress: System Integration & Deployment

The final week focused on merging the three distinct modules into a cohesive system and deploying it for public access.

5.1 System Integration (Krish)

Krish connected the frontend to the backend services. The **AI Model** was wrapped in a Flask API to serve predictions to the frontend. The **IoT Data** stream was connected to the dashboard using Fetch API for real-time updates without page reloads. The complete application was deployed using **Vercel** for the frontend and **Render** for the backend Python services.

5.2 IoT Connectivity (Krishna)

Krishna integrated the ESP8266 Wi-Fi module to connect to the internet. He implemented HTTP POST requests to send the JSON formatted sensor data directly to the hosted backend API. A "Real-Time Monitoring" test was conducted where changes in sensor conditions (e.g., watering the plant) were reflected on the web dashboard within 2 seconds.

5.3 Model Deployment (Abhilasha)

Abhilasha saved the trained Keras model as an `.h5` file and optimized it for inference speed. She developed a Python script to handle image preprocessing on the server side before passing it to the model. The prediction API was tested with real crop images captured via mobile phone to ensure robustness against varying lighting conditions.

5.4 Final System Capabilities

- **Dashboard:** Displays live temperature, humidity, and moisture.
- **Disease Scanner:** Users upload a photo -> System returns disease name + confidence score.
- **Alerts:** Visual alerts if moisture drops below critical thresholds.

Technical Architecture & System Design

6.1 System Architecture Overview

The SmartKisan AI system follows a classic **Three-Tier Architecture**:

1. **Presentation Layer (Frontend):** The user interface accessible via web browsers.
2. **Logic Layer (Backend & AI):** Processes business logic, handles API requests, and runs ML inference.
3. **Data Layer (Database & IoT):** Stores historical data and generates real-time environmental data.

6.2 Data Flow Diagram

```
[ IoT Sensors ] ----> [ Microcontroller (ESP8266) ] | v (HTTP / JSON)
  | v [ User Client ] <---> [ Backend API Server ] <---> [ AI Model
                        Engine ] ^ | [ Database ]
```

6.3 Technology Stack

Component	Technology Used
Hardware	ESP8266 NodeMCU, DHT11, Capacitive Soil Moisture Sensor
Firmware	C++ (Arduino Framework)
Frontend	HTML5, CSS3, JavaScript (Vanilla)
Backend	Python (Flask Framework)

AI / ML	TensorFlow, Keras, OpenCV
Deployment	Vercel (Frontend), Render (Backend)

6.4 API Structure

The system relies on two primary API endpoints:

- `POST /api/predict` : Accepts an image file, preprocesses it, and returns the disease class prediction.
- `POST /api/telemetry` : Accepts JSON payload `{temp, humidity, moisture}` from the IoT device and updates the database.
- `GET /api/status` : Returns the latest sensor readings for the frontend dashboard.

Challenges Faced & Solutions

During the development lifecycle, the team encountered several technical and logical hurdles. Below is a summary of the key challenges and the engineering solutions applied.

7.1 Sensor Calibration Instability

Problem: The soil moisture sensor initially provided fluctuating readings. Even in constant soil conditions, values would jump by +/- 15%, making the data unreliable for decision making.

Solution: We implemented a software filter (Moving Average Filter) in the microcontroller code. Instead of sending raw values, the system takes 10 readings over 2 seconds, averages them, and maps the result to a 0-100% scale using pre-measured dry air and submerged water values.

7.2 Low Model Accuracy on Real Images

Problem: While the AI model performed well on the training dataset (95% accuracy), it failed when testing with photos taken from mobile cameras in the field (dropping to 60%). This was due to differences in lighting and background noise.

Solution: We utilized **Data Augmentation** techniques. We artificially expanded the training dataset by adding rotated, zoomed, brightness-adjusted, and noisy versions of the original images. This forced the model to learn robust features independent of lighting conditions, improving real-world accuracy to 89%.

7.3 CORS & API Connectivity

Problem: Connecting the frontend hosted on Vercel with the backend hosted on a different domain caused Cross-Origin Resource Sharing (CORS) errors, blocking data flow.

Solution: We configured the Flask backend to include specific CORS headers (`Access-Control-Allow-Origin`) allowing requests specifically from our frontend domain, ensuring secure and functional communication.

7.4 Hardware Power Consumption

Problem: The ESP8266 Wi-Fi module consumed significant power, which is not ideal for battery-operated field devices.

Solution: We implemented `Deep Sleep` mode in the microcontroller. The device now wakes up, reads data, transmits it, and goes back to sleep, significantly extending potential battery life.

Final System Outcome & Features

The project successfully culminated in a working prototype of the SmartKisan AI platform. The system is robust, responsive, and meets the initial objectives set out in the planning phase.

8.1 Complete Feature List

- **Live Environment Dashboard:** Real-time gauges for Humidity, Temperature, and Soil Moisture.
- **AI Disease Doctor:** Drag-and-drop interface for leaf analysis with instant results.
- **Advisory System:** Provides specific chemical or organic treatment suggestions based on the detected disease.
- **Historical Logs:** Keeps a record of past scans and sensor readings (Beta).

8.2 Performance Metrics

Metric	Result
AI Prediction Accuracy	92.4% (Validation Set)
IoT Data Latency	~1.5 Seconds
Sensor Error Margin	± 2%
API Response Time	< 300ms

8.3 System Demonstration

The final system allows a user to place the IoT device in a plant pot and immediately see the moisture levels on their laptop screen. When a leaf showing signs of "Late Blight" was

uploaded to the system, the AI correctly identified the pathogen and recommended a fungicide treatment plan.

User Interface Highlights

The UI was specifically designed with high contrast and large buttons to accommodate usage on mobile screens in bright sunlight, a common scenario for farmers.

Conclusion & Future Enhancements

9.1 Project Achievements

The SmartKisan AI project successfully demonstrates the convergence of IoT and AI in agriculture. We have built a system that not only monitors the "health" of the environment via sensors but also the "health" of the plant via computer vision. The project highlights how accessible technology can solve critical problems in food security and farming efficiency.

9.2 Learning Outcomes

Through this project, the team gained hands-on experience in:

- **Full Stack Integration:** Connecting disparate technologies into a single workflow.
- **Embedded Systems:** Real-world sensor calibration and power management.
- **Deep Learning:** Practical challenges of deploying Neural Networks in production environments.

9.3 Future Enhancements

To make this system a viable commercial product, the following improvements are proposed:

1. **Mobile Application:** Developing a native Android app for offline capabilities.
2. **Expanded Sensor Array:** Adding NPK (Nitrogen, Phosphorus, Potassium) sensors and pH sensors for comprehensive soil analysis.
3. **Edge AI:** Running the lighter version of the AI model directly on the Raspberry Pi or mobile device to remove internet dependency.

4. **Drone Integration:** Using drones to capture field images automatically for large-scale analysis.

9.4 Impact on Agriculture

SmartKisan AI has the potential to democratize precision agriculture. By reducing the cost of entry for smart farming tools, we can help small-scale farmers increase their yield, reduce chemical usage, and ultimately improve their economic standing.

Acknowledgments

We would like to thank our faculty mentors and the open-source community for providing the tools and datasets that made this project possible.